

Software de Orquestación

Terraform / OpenTofu

✉ José Domingo Muñoz

🏫 IES Gonzalo Nazareno · Dos Hermanas

📖 PI · Proyecto Intermodular

PI · UNIDAD 1 — SOFTWARE DE ORQUESTACIÓN

01

Infraestructura como Código

Qué es y por qué la usamos

Del despliegue manual al código

DESPLIEGUE TRADICIONAL

- Configuración manual, servidor a servidor
- **Difícil de reproducir** y de documentar
- La documentación se desactualiza o no existe
- *"Funciona en mi máquina"*

CON INFRAESTRUCTURA COMO CÓDIGO

- La infraestructura se **describe en ficheros de texto**
- Se crea y se modifica de forma **automática**
- Es **reproducible**: el mismo código da siempre el mismo resultado
- Se gestiona con **Git**, igual que el software

¿Qué es la Infraestructura como Código (IaC)?

" *Consiste en **gestionar y aprovisionar infraestructura** (servidores, redes, almacenamiento...) mediante **ficheros de código**, en lugar de configurarla manualmente.*

LO HACE POSIBLE

Los servicios de virtualización y Cloud Computing son **software**, y todo software se puede **programar mediante una API**.

BENEFICIOS

- Escenarios **replicables** y predecibles
- **Control de versiones** de toda la infraestructura
- **Auditoría** de los cambios y recuperación rápida ante desastres

Dos tipos de herramientas IaC



SOFTWARE DE ORQUESTACIÓN

Crea y gestiona **escenarios completos**:
máquinas virtuales, redes, almacenamiento...

Ejemplos: Terraform, OpenTofu, Pulumi,
CloudFormation, Heat



SOFTWARE DE GESTIÓN DE LA CONFIGURACIÓN (CMS)

Configura el **software** de las máquinas ya
creadas.

Ejemplos: Ansible, Puppet, Chef, Salt



En este módulo usamos **OpenTofu** para orquestar y **Ansible** para configurar: primero se crea la infraestructura, después se configura.

02

Software de orquestación: OpenTofu

Orquestación de infraestructura declarativa

De Terraform a OpenTofu

TERRAFORM

Herramienta de orquestación creada por **HashiCorp** (2014), lenguaje declarativo propio **HCL**. En agosto de 2023 cambió su licencia a **BUSL 1.1**, que ya no es software libre.

OPENTOFU

Fork libre de Terraform 1.5.x creado por la comunidad y gobernado por la **Linux Foundation**. Licencia **MPL 2.0** (libre), compatible con los ficheros `.tf` y con los providers de Terraform.

💡 Usamos **OpenTofu** en lugar de Terraform precisamente por ser software libre, sin restricciones de uso.

Elementos de configuración

LOS CUATRO BLOQUES BÁSICOS

- **Provider:** plugin que conecta con una API concreta
- **Resource:** elemento de infraestructura a crear (VM, red, volumen...)
- **Variable:** parámetro para reutilizar la configuración
- **Output:** valor de salida (IP, ID...) tras aplicar el plan

EJEMPLO

```
variable "memoria" { default = 1024 }

resource "libvirt_domain" "web" {
  name     = "web"
  memory   = var.memoria
}

output "nombre" {
  value = libvirt_domain.web.name
}
```

Un mismo lenguaje, distintos providers

Un **provider** es un plugin que conecta OpenTofu con una plataforma concreta. Cada uno define sus propios *resources*, pero el lenguaje (HCL) es siempre el mismo — en este módulo usaremos `libvirt` y `openstack`.

PROVIDER	PLATAFORMA
<code>aws</code>	Amazon Web Services
<code>azurerm</code>	Microsoft Azure
<code>google</code>	Google Cloud Platform
<code>openstack</code>	Nube privada OpenStack
<code>libvirt</code>	Máquinas virtuales con KVM
<code>docker</code>	Contenedores Docker

Ficheros de un proyecto

LOS ESCRIBIMOS NOSOTROS

- `provider.tf` — el provider y su versión
- `variables.tf` — las variables reutilizables
- `main.tf` — los recursos: VMs, discos...
- `network.tf` — las redes
- `output.tf` — lo que se muestra al terminar (IPs...)
- `cloud-init/` — la configuración inicial de cada VM

LO GENERA OPENTOFU

- `terraform.tfstate` — el **estado** de la infraestructura
- `.terraform/` — providers descargados por `init`
- `.terraform.lock.hcl` — versiones exactas de los providers

No se editan a mano ni se suben a Git (`.gitignore`).

Dependencias entre recursos

```
resource "libvirt_volume" "disco" {  
  name = "web.qcow2"  
  ...  
}  
  
resource "libvirt_domain" "web" {  
  disk { volume_id = libvirt_volume.disco.id } # referencia a otro recurso  
}
```

- Al usar `libvirt_volume.disco.id`, OpenTofu sabe que **el disco va antes** que la máquina
- Con esas referencias construye un **grafo de dependencias**: crea en orden, lo independiente a la vez, y **destruye al revés**
- No importa en qué orden se escriban los recursos ni en qué fichero: se leen todos los `.tf` del directorio

El estado de la infraestructura

" El **estado** (`terraform.tfstate`) es un fichero donde OpenTofu guarda **qué recursos ha creado realmente** y con **qué identificadores**.

¿POR QUÉ SE GUARDA?

- Es la forma de saber **qué existe ya** sin preguntar a la plataforma recurso a recurso
- Permite **comparar** lo declarado en el código con lo que hay realmente creado

CUIDADO CON EL ESTADO

- **No se edita a mano**
- Si trabajan varios a la vez, hace falta un **backend remoto** compartido
- Si el estado se pierde, OpenTofu "olvida" lo que había creado

Comandos más útiles

COMANDO	DESCRIPCIÓN
<code>tofu init</code>	Inicializa el proyecto, descarga los <i>providers</i>
<code>tofu plan</code>	Muestra qué acciones se realizarán (sin aplicar)
<code>tofu apply</code>	Aplica los cambios: crea, modifica o elimina recursos
<code>tofu destroy</code>	Elimina todos los recursos gestionados por el proyecto
<code>tofu validate</code>	Verifica que la sintaxis de los ficheros <code>.tf</code> sea correcta
<code>tofu show</code>	Muestra el estado actual de los recursos creados
<code>tofu output</code>	Muestra los valores definidos en los bloques <code>output</code>
<code>tofu refresh</code>	Actualiza el estado con lo que hay realmente en la plataforma

Flujo de trabajo habitual

```
tofu init      # 1. Una sola vez: descarga el provider
tofu plan      # 2. Revisar qué se va a crear, cambiar o borrar
tofu apply     # 3. Crear el escenario (pide confirmación)
tofu output    # 4. Ver la información del output (IPs...)
tofu destroy   # 5. Eliminar todo el escenario
```

- Si cambias los ficheros, vuelve a `plan` y `apply`: OpenTofu solo aplica **la diferencia**
- Todos los comandos se ejecutan **en el directorio del proyecto**

¿Qué hace realmente `tofu plan`?

" `plan` *compara tres cosas y muestra la diferencia, sin modificar nada todavía.*

1. EL CÓDIGO

Lo que **declaras** en los ficheros `.tf` (el estado deseado)

2. EL ESTADO

Lo último que OpenTofu **sabe** que existe
(`terraform.tfstate`)

3. LA REALIDAD

Lo que **hay de verdad** en la plataforma (puede haber cambiado a mano: *drift*)

💡 `plan` es la red de seguridad antes de aplicar: revisa siempre su salida antes de ejecutar `apply`.

03

OpenTofu + libvirt

Máquinas virtuales con KVM y cloud-init

Los ejemplos del repositorio `ejercicios_pi`

Cada ejemplo está en su directorio `opentofu/ejemploN` y sus recursos llevan el prefijo `ejN-`:

EJEMPLO	ESCENARIO
<code>ejemplo1</code>	1 máquina Debian en la red <code>default</code>
<code>ejemplo2</code>	Igual, con un disco adicional de 1 GB
<code>ejemplo3</code>	1 máquina conectada a dos redes con DHCP : una NAT creada por OpenTofu y <code>default</code>
<code>ejemplo4</code>	Red NAT con DHCP + red aislada sin DHCP (IP estática)
<code>ejemplo5</code>	2 máquinas (Debian y Ubuntu), red muy aislada e inventario de Ansible

- **Destruye siempre el escenario** antes de pasar al siguiente: varios usan el mismo rango (`192.168.100.0/24`)

El provider `libvirt`

```
terraform {  
  required_providers {  
    libvirt = {  
      source  = "dmacvicar/libvirt"  
      version = "0.8.3"  
    }  
  }  
}  
  
provider "libvirt" {  
  uri = "qemu:///system"  
}
```

- `uri` — la conexión a libvirt, la misma que `virsh -c qemu:///system`
- Versión **fijada** a la `0.8.3` (la `0.9` usa otra sintaxis): consulta **su** documentación
- `provider.tf` y `output.tf` **no hay que modificarlos**

Imágenes cloud y el pool

Las **imágenes cloud** son sistemas ya instalados, mínimos, en formato `qcow2` y preparados para configurarse con cloud-init en el primer arranque.

```
cd /var/lib/libvirt/images          # directorio del pool "default"
sudo wget <url de la imagen> -O debian13-base.qcow2
sudo qemu-img resize debian13-base.qcow2 10G
sudo virsh pool-refresh default     # que libvirt vea el fichero nuevo
```

- Se descargan **una vez** y se usan como **imagen base** de todas las máquinas
- En los ejemplos: `debian13-base.qcow2` (Debian 13) y `ubuntu2604-base.qcow2` (Ubuntu 26.04)
- Se eligen con variables de `variables.tf`: `var.base_image` y `var.libvirt_pool_name`

Los discos: `libvirt_volume`

Clon ligero de la imagen base (*backing store*: solo guarda los cambios) y disco extra vacío de 1 GB:

```
resource "libvirt_volume" "ej2-server1-disk" {
  name           = "ej2-server1.qcow2"
  pool           = var.libvirt_pool_name
  base_volume_name = var.base_image
  base_volume_pool = var.libvirt_pool_name
  format         = "qcow2"
}

resource "libvirt_volume" "ej2-server1-disk-extra1" {
  name     = "ej2-server1-disk-extra1.qcow2"
  pool     = var.libvirt_pool_name
  format   = "qcow2"
  size     = 1 * 1024 * 1024 * 1024 # 1 GB en bytes
}
```

cloud-init

Configura la máquina en su **primer arranque**, a partir de dos ficheros:

user-data : usuarios, claves, paquetes...

```
#cloud-config
hostname: ej4-server1
timezone: Europe/Madrid
users:
  - name: debian
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-ed25519 AAAA... tu clave
package_update: true
package_upgrade: true
```

network-config : la red (netplan)

```
network:
  version: 2
  ethernet:
    ens3:
      dhcp4: true
    ens4:
      dhcp4: false
      addresses: ["192.168.130.10/24"]
```

- Se empaquetan en una **ISO** con el recurso **libvirt_cloudinit_disk** (parámetros **user_data** y **network_config**)

La máquina virtual: `libvirt_domain`

```
resource "libvirt_domain" "ej1-server1" {  
  name      = "ej1-server1"  
  memory    = 1024  
  vcpu      = 2  
  network_interface {  
    network_name      = "default"  
    wait_for_lease    = true  
  }  
  disk { volume_id = libvirt_volume.ej1-server1-disk.id }  
  cloudinit = libvirt_cloudinit_disk.ej1-server1-cloudinit.id  
  console {  
    type          = "pty"  
    target_port    = "0"  
    target_type    = "serial"  
  }  
}
```

- `console`: consola serie, la esperan las imágenes cloud y permite `virsh console` aunque falle la red

Redes de libvirt

TIPO	DEFINICIÓN	¿EL ANFITRIÓN TIENE IP?	¿SALIDA AL EXTERIOR?
NAT	mode = "nat" + addresses	Sí (la .1)	Sí
Aislada	mode = "none" + addresses	Sí (la .1)	No
Muy aislada	mode = "none" , sin addresses	No	No

```
# network.tf: la red se crea con "tofu apply" y se elimina con "tofu destroy"
resource "libvirt_network" "ej3-nat-dhcp" {
  name      = "ej3-nat-dhcp"
  mode      = "nat"
  addresses = ["192.168.100.0/24"]
  dhcp { enabled = true }
  autostart = true
}
```

Conectar la máquina a las redes

RED QUE NO GESTIONA OPENTOFU

Por su **nombre** (por ejemplo, `default`):

```
network_interface {  
  network_name = "default"  
  wait_for_lease = true  
}
```

RED CREADA POR OPENTOFU

Por su **id**:

```
network_interface {  
  network_id = libvirt_network  
    .ej4-aislada-static.id  
}
```

- `wait_for_lease = true` — esperar a que el DHCP le dé IP: **solo** en redes con DHCP (sin DHCP, el `apply` se queda esperando)
- Cada `network_interface` es una interfaz más (`ens3`, `ens4` ...), que **hay que configurar** en el `network-config`

Las IP en el `output`

```
output "ej4-server1" {  
  value = {  
    nombre = "ej4-server1"  
    ip1     = try(libvirt_domain.ej4-server1.network_interface[0].addresses[0], "No disponible")  
    ip2     = "192.168.130.10" # estática: está en cloud-init/network-config1.yaml  
  }  
}
```

- Con `wait_for_lease`, OpenTofu lee la IP de las **concesiones** (*leases*) del DHCP de libvirt
- Las IP **estáticas** OpenTofu no las conoce: se escriben en el `output`, las mismas que en el `network-config`
- `try(..., "No disponible")` — si no hay IP, el `output` no da error
- Sin `wait_for_lease`, `apply` termina en cuanto arranca la máquina, todavía sin IP: «No disponible»

Ciclo de vida de los recursos

Si cambiamos un escenario que ya existe, `tofu plan` indica qué hará con cada recurso:

SÍMBOLO	ACCIÓN	EJEMPLO
~	update in-place: se modifica sin destruirlo	Añadir <code>autostart = true</code> a la máquina
-/+	destroy and then create replacement: se recrea (<code>forces replacement</code>)	Cambiar la memoria · cambiar <code>var.base_image</code>
+	create: se crea	Una máquina borrada a mano con <code>virsh</code>

💡 Recrear una máquina es empezar de cero: se pierde lo que había en su disco y puede cambiar su IP. Los cambios se **propagan**: si se recrea el disco, también se recrea la máquina que lo usa.

Del `output` al inventario de Ansible

`inventario.tf`: genera el fichero `hosts` con las IP del escenario

```
resource "local_file" "inventario" {
  filename      = "${path.module}/hosts"
  file_permission = "0644"
  content = templatefile("${path.module}/inventario.tftpl", {
    ip_server1 = try(libvirt_domain.ej5-server1.network_interface[0].addresses[0], "")
    ip_server2 = "10.0.0.2" # estática: está en cloud-init/network-config2.yaml
  })
}
```

- `templatefile` rellena la plantilla con las IP · `local_file` la escribe en tu equipo
- Usa el provider `hashicorp/local`: hay que añadirlo en `provider.tf` y repetir `tofu init`
- Cada `apply` genera el inventario con las IP correctas: **primero OpenTofu, después Ansible**

La plantilla del inventario

`inventario.tftpl`: cada `${...}` se sustituye por el valor que le pasa `templatefile`

```
# Fichero generado por OpenTofu (inventario.tf): no lo edites a mano
[servidores]
ej5-server1 ansible_host=${ip_server1} ansible_user=debian
# server2 solo es accesible a través de server1 (ProxyJump)
ej5-server2 ansible_host=${ip_server2} ansible_user=ubuntu ansible_ssh_common_args='-o ProxyJump=debian@${ip_server1}'
```

- `ej5-server1` (Debian): IP por DHCP en la red NAT y `10.0.0.1` en la red muy aislada
- `ej5-server2` (Ubuntu): solo está en la red muy aislada (`10.0.0.2`), así que Ansible llega a ella **a través de** `ej5-server1` (*ProxyJump*)
- `ej5-server2` **no tiene salida al exterior**: en su `user-data` está comentada la actualización de paquetes

PI · UNIDAD 1 — SOFTWARE DE ORQUESTACIÓN

¡Gracias!

Software de Orquestación · OpenTofu

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI